SOURCE SECURITY REVIEW

Blast launch stack

Fable audit with the Cloudflare security-audit skill

Audit model
Claude Fable 5.1 - claude-fable-5-1

Effort
Initially xhigh ("extra"); high for remaining work, at the user's request

Execution
Anthropic API (continuations 1-5, orchestrated by Codex); in-session Claude Code subagents on the user's Claude plan (continuation 6, orchestrated by Claude Opus 5.5, which made no security judgements)

Reviewed revision
d80f2834468ba798df66f36d01fd64eec9d4df91

Audit skill
cloudflare/security-audit-skill

Skill revision
c1c8a8c1471069fb0e188eeaff69b8e8db6564a8

Scope
sui/admin, sui/core, sui/migrators, sui/vault, sui/launchpad, sui/presale

Date
6 November 2026

Run status
Complete scoped source review

Reading this report

Fable produced every audit conclusion, every verification and the three reports using the requested skill. Codex orchestrated continuations 1-5. Claude Code (Claude Opus 5.5) orchestrated continuation 6 and rendered this PDF from the Fable-authored Markdown without changing it. Every continuation-6 transcript was checked to contain only claude-fable-5-1 responses. One discarded critic attempt and one disclosed tool deviation are described in the report. The audit is source-only: runtime and deployed behavior have not been tested. Read the coverage and validation limits alongside the findings.

Reviewed source | Pinned audit skill | Anthropic effort documentation

Blast V2 launch stack: security audit report

Cloudflare `security-audit` skill run `2026-11-06-fable-cloudflare-launch-stack`, performed by Claude Fable 5.1 (`claude-fable-5-1`) on the production Move contract code of `interest-protocol/blast-v2-contracts` at `d80f2834468ba798df66f36d01fd64eec9d4df91`.

Result in one line: **zero confirmed findings, one lead that needs validation, one candidate rejected on source evidence.** This was a scoped, source-only run; nothing was built, run, tested or probed, so no record could reach `confirmed` and runtime and deployment behaviour are untested. Read it as a partial pass over the contract source, not as a clean bill for the deployed system.

1. Run profile, scope and execution

Item	Value
Target repository	`interest-protocol/blast-v2-contracts`
Source ref	`d80f2834468ba798df66f36d01fd64eec9d4df91` (`main`, fetched 2026-11-06; unmodified git-archive snapshot with no `.git` directory; integrity manifest of 245 scoped files)
Skill	`cloudflare/security-audit-skill` at `c1c8a8c1471069fb0e188eeaff69b8e8db6564a8`
Profile	`standard`, scoped, no budget set
Scope paths	`sui/admin`, `sui/core`, `sui/migrators`, `sui/vault`, `sui/launchpad`, `sui/presale`
Effective scope (user rule)	Production Move contract source in the six packages: `sources/**/*.move`, the production interface stubs the migrators link against, and linked contract dependency behaviour. "We want to audit the code not our harnesses"; "focus on move code not harness etc"
Execution policy	`sandboxed-source-and-local-only`, run in `source-only` mode (see 1.3)
Model	`claude-fable-5-1` for every reconnaissance, hunting, critic, candidate-validation, final-verification and report-authoring call
Effort	`xhigh` (the user's word: "extra") for the coordinator, the 4 reconnaissance calls and the first 12 hunters; changed by the user to `high` for the 12 later hunters, both coverage critics, both Phase 3 verifiers, both Phase 5 verifiers and this report
Dependency reference	Pinned Sui framework `coin_registry.move` at `06734f6ff0af45d8632a14a4dc4b100197f6b1a2` (the revision `sui/migrators/Move.toml` pins), fetched read-only outside the snapshot for the Bluefin candidate's framework premise; not executed
Prior run	2026-11-05 narrative Fable deep audit of older source `4fe8d739`; prior evidence only, no compatible ledger

1.1 Agents spent

No budget was set. Wave 1 planned 21 hunters; the run spent 24 hunter calls because units were reopened after the provider failures and six units were superseded after the user's scope correction.

Role	Count	Model / effort
Reconnaissance	4	Fable / xhigh
Hunters	24	Fable; 12 at xhigh, 12 at high
Coverage critics (accepted)	2	Fable / high (post-wave critic and final-clean critic)

Role	Count	Model / effort
Coverage critic (discarded)	1	see 1.4
Phase 3 candidate verifiers	2	Fable / high
Phase 5 final-record verifiers	2	Fable / high
Phase 6 report writer	1	Fable / high

1.2 Execution channel

• Continuations 1 to 5 ran Fable through the Anthropic API, orchestrated by Codex. The run was interrupted three times by the provider: a credit failure at 2026-11-06T10:51:18Z after the 4 reconnaissance and first 12 hunter returns; a billing rejection of continuation 4 before any source work; and an API usage limit at 2026-11-06T15:47:17Z after hunters 21 and 22. Between those, continuations 2 and 3 completed hunters 13 to 20 and were then deliberately interrupted so the user's scope correction could be applied (the two live delegates returned nothing, and no evidence from them is claimed). Continuation 5 applied the scope correction and completed hunters 21 and 22.

• Continuation 6 (after the usage limit) ran Fable as in-session Claude Code subagents on the user's Claude plan. Claude Code (Claude Opus 5.5) orchestrated only: prompt assembly, ledger bookkeeping, the two JSON validators, evidence capture and PDF rendering. It made no security judgements. Every continuation-6 transcript was checked to contain only `claude-fable-5-1` responses. Continuation 6 completed hunters 23 and 24, both coverage critics, both Phase 3 verifiers, both Phase 5 verifiers and this report.

• No model other than Fable performed security analysis at any point. Prior completed calls keep their xhigh provenance; later calls are high.

1.3 Source-only limits: why this is a partial pass

OS-enforced sandbox controls (empty allowlisted environment, no-network namespace, read-only target and toolchain mounts, scratch-only writes, resource limits) and race-safe artifact promotion were not provisioned for this run. Therefore no target build, `sui move test`, `cargo test`, script, fixture, fuzzer, network call, chain RPC or mainnet ABI probe was executed. Every ledger check is `method: "source"` with `artifact: null`. The only Bash permitted to the coordinator was the skill's two JSON validators.

Consequences, stated plainly:

• No record can reach verdict `confirmed`. A source-grounded candidate that survived refutation is `needs_validation` with the missing runtime gate recorded as a blocker; a candidate that source refutes is `rejected`.

• Runtime and deployment behaviour (published package IDs, cap custody, admitted sets, raised notices, venue bytecode behaviour, Quote deny lists, Merkle manifests, signer custody) are untested.

• "Complete" below means only that every phase of this scoped, source-only workflow ran to its end (Phases 1 to 6, both critics, both verification phases). It does not mean the system was exercised.

1.4 Integrity deviations disclosed

1. One final-clean coverage critic attempt delegated an inventory step to a nested agent that ran on Claude Opus 5.5 and used Bash (read-only). That violates the Fable-only analysis rule and the read-only, no-Bash critic contract. Its return was discarded, its transcripts retained as `*-DISCARDED.jsonl`, and the final-clean critic was re-run fresh.

2. The accepted final-clean critic (`c6-final-clean-critic-2`) used read-only Bash lookups 35 times (`find`, `grep`, `sed`, `awk` over target source and audit JSON, and one read-only `python3` read of the parent ledger view) despite

a Read/Grep/Glob-only instruction. Every turn was `claude-fable-5-1`, nothing was written, no target code was executed and nothing was delegated, so the return was accepted with this disclosure.

This report was derived only from the final verified records, the coverage ledger, the hunter hardening notes and the run metadata, using file reads only.

1.5 Prior-run use, carried confirmations and revalidations

The 2026-11-05 narrative Fable deep audit of the same six packages at `4fe8d739` has no `coverage-ledger.json` or `findings.json` in this skill's format, so there is no compatible prior ledger. Every unit was seeded `prior_status: "none"`. Prior items (vault unpause version gate, treasury guard coverage in unpause, auto-LP compounding path, holder-reward beneficiary addresses, Bluefin auto-LP estimate) were re-compared against current source during reconnaissance and used only as priority input. There are **no carried same-source confirmations and no changed-source revalidations**.

1.6 Deferred and out-of-scope coverage

- **Deferred at the end of the run: none.** 25 units were deferred mid-run solely for the first credit failure and reopened on continuation 2; 3 units were deferred at the usage limit and completed on continuation 6.
- **Blocked: none.**
- **Out of scope (7 ledger units, not counted as covered):**
  - Six repository-hygiene "Obvious things" units, one per package (`Move.toml, Move.lock, sources, tests, scripts` / `repository hygiene: hardcoded addresses and secrets, security TODO/FIXME, test-only gating, dependency pins, publish scripts`). They were superseded on continuation 5 under the user's scope correction and replaced by six production-contract units (`<package>/sources#production Move modules` / `obvious reachable contract defects`), which were hunted. The old-to-new mapping is recorded per unit (`superseded_by`). No harness or tooling coverage is claimed.
  - One external dependency item: `sui/vesting/Published.toml#mainnet pin versus sui/vesting/sources` against `sui/core/sources/accounting/payout.move#pay_or_vest reliance on published blast_fun_vesting new_irrevocable checks`. `sui/vesting` is outside the six scoped packages and the decisive fact (whether the pinned mainnet package matches the source at this ref and has no live `UpgradeCap`) is a deployed-bytecode fact no source-only check can establish. The ledger records the owner-observed check: `sui client verify-source` for the pinned package against `sui/vesting` at this ref, plus confirming there is no live `UpgradeCap`.
- A git-history review step could not be applied because the snapshot is a git archive with no `.git` directory.

1.7 Scope rule applied to the prose

Per the user's rule, every hunter hardening note, observation or recommendation whose subject is a harness, test, `ptb_tests`, fixture, script, Cargo or toolchain, CI, ABI checker, lockfile or repository hygiene, or publishing machinery was excluded from this report (for example: fork-snapshot age, `Move.lock` override nodes, `check_mainnet_abi.mjs` coverage, `check_source_boundaries.sh` pins, fixture signer seeds, requests for new harness rows or unit tests). Notes about production contract code or its linked dependency behaviour were kept; where a record cites tests or docs as supporting evidence, the citation is kept as evidence only.

2. Security posture summary

Across 55 effective in-scope contract units reviewed from source, no lower-trust path was found that lets a party take or permanently lock another party's funds, mint or inflate supply, bypass a role check, or act on a sale, vault or round it does not own. The strongest controls hold on source: the live-table `blast_acl::sign_in` role check, `with_original_ids` witness comparison for every cross-package seal, no-ability hot potatoes for migration and ownership proofs, stored cap-ID binding, `derived_object::claim` uniqueness, compiled economic ceilings, and the version-gate, pause-bit, lifecycle check ordering. The one surviving lead is an operator-triggered availability and recovery hazard, not a theft path: the vault's version advance is not coupled to the dependent migrators relink, and during that venue outage any sender can fire the presale's designed refund exit once a Funded sale's fixed liquidity deadline passes. Its runtime premises are untested. The hardening notes cluster around the same operational seams: upgrade sequencing between coupled packages, delayed settings whose post-release delay may be lowered to zero, type-origin heuristics in venue and witness admission, and a few uniformity gaps in version gating and creation-time validation.

3. Confirmed findings

Severity	Title	Affected boundary	Observed result
—	None	—	No record reached `confirmed`; see 1.3

4. Confirmed finding details

None. `FINDINGS-DETAIL.md` states the same and contains no reproductions.

5. NEEDS VALIDATION

One lead. It carries **no severity** and is **not a confirmed vulnerability**. Full source trace, evidence, blockers and plans are in `NEEDS-VALIDATION.md`.

Lead	Repository trace	Exact blocker	Bounded local next step	Safe owner-observed deployment check
NV-1. Vault version advance is uncoupled from the migrators relink; during the resulting venue outage any sender can irreversibly cancel a Funded presale whose liquidity deadline passes. Fingerprint `blast-v2/vault/vault/venue-gate-version-advance-uncoupled-from-migrators-relink`	Entry `sui/presale/sources/blast_presale.move:851` (`cancel_unlaunched`: version, Funded, deadline only; no pause bit) → `sui/admin/sources/package/package_admin.move:165` (`commit_upgrade_and_advance<BLAST_VAULT_PACKAGE>` advances the shared version at `:190` in the same call; no cross-package check) → `sui/admin/sources/package/package_version.move:86` (`assert_current` strict equality; `advance` only increments) → `sui/vault/sources/vault.move:128` (every venue-only mutator inlines `constants::version!()`, literal 1) → `sui/migrators/source`	(1) No sandboxed `cargo test` or `sui move test` was permitted, so the `EVersionMismatch` abort of `cetus/bluefin migrate` and `collect_fees` after the vault commit, and recovery only after a relinked migrators upgrade, were not observed. (2) The Sui rule that a dependent keeps calling the dependency generation it was published against is a framework fact outside the repository. (3) Whether the relink can be scheduled before the vault upgrade executes (precomputed post-upgrade package ID, same-transaction dependency) is a	Offline, loopback-only simulacrum run in `sui/launchpad/ptb_tests`: publish the stack, open a presale with the 7-day minimum liquidity window, fund and settle it; perform a compatible vault upgrade with the vault version literal rewritten to 2; assert `presale.migrate` + venue `migrate` rolls back with `EVersionMismatch` while `prepare` still succeeds; measure whether the relink digest can be included in the vault commit transaction; set all presale pause bits, advance to the deadline, assert an unrelated sender's `cancel_unlaunched` succeeds and `migrate`	Read-only, from release scripts and records: whether a vault upgrade is ever executed before the relinked migrators digest is scheduled; how the relink digest is produced; the migrators' applied upgrade notice at release time; which AdminCaps hold `CancelAdmin<BLAST_MIGRATORS>`; that no Funded presale is within its last (notice + build time) of its liquidity window at the planned vault commit; or whether the vault is to be made immutable before any upgrade

Lead	Repository trace	Exact blocker	Bounded local next step	Safe owner-observed deployment check
	s/venues/cetus.move:170 and bluefin.move:125 (venues reach the vault only through the generation they link; prepare never touches the vault) → sui/presale/sources/blast_presale.move:924 (migrate returns a no-ability Migration; deadline fixed at share, no extension) → sink blast_presale.move:1662 (cancel is one-way; refunds in full; minted supply stays locked)	toolchain/runtime fact outside source. (4) Deployment facts: the operators' actual vault-release script, whether the migrators notice is pre-lowered, who holds CancelAdmin<BLAST_MIGRATORS>, whether any Funded presale's deadline falls inside the window	then aborts EInvalidLifecycle; schedule, cancel and reschedule the relink; commit the relink and assert migrate succeeds for a still-Funded sale	

6. Hardening notes and positive source patterns

Hardening notes are hunter observations retained in ledger bookkeeping. None is a finding; none carries severity; many are explicitly "not a finding" or "same-principal". Duplicates from several hunters were merged. Reporting hunter IDs are in parentheses. Paths are repository-relative at d80f2834.

6.1 Upgrade and version-gate sequencing across coupled packages

Context for NV-1 and independent of it:

- vault::new is version-gated (sui/vault/sources/vault.move:128) and reached only from the migrators package (cetus.move:170, bluefin.move:125); advancing the vault version before the migrators package is relinked makes every venue migrate abort until the migrators upgrade lands (w1-launch-migrate, w2-upgrade-linkage).
- cetus::prepare and bluefin::prepare stay callable during a vault-relink window because they check only the migrators version (cetus.move:71, bluefin.move:47), so new sales keep opening against venues that cannot migrate; consider setting each product's create pause bit for the duration of a vault release (w2-upgrade-linkage).
- The runbooks (sui/migrators/DEPENDENCIES.md:524-531, sui/migrators/DEPLOYING.md:95-97, deployments/README.md:118-124) should state how "schedule both together" is achieved, since the relink's upgrade digest covers the vault's post-upgrade package ID: either document deterministic precomputation of the new vault ID or a temporary, reviewed reduction of the migrators notice, and schedule vault releases outside any Funded presale's final days (w2-upgrade-linkage).
- CancelAdmin<BLAST_MIGRATORS> can cancel a ready relink and restart the notice while venue routes are down; bounded by immediate SuperAdminCap revocation (sui/admin/sources/blast_acl.move:112-123) but worth naming as a liveness-critical role during vault releases (w2-upgrade-linkage).
- commit_upgrade_and_advance always advances PackageVersion by one, so every upgrade must bump the consumer's compiled version constant; a mismatch strands every assert_current gate for at least the applied delay until a repair upgrade lands. For the vault that includes direct exits (claim_owner_balances, holder_rewards::claim, pay_protocol_fees_with_partner); package_admin itself is never version gated (package_admin.move:14-16) so repair is always schedulable. A pre-commit dry run against the new bytecode's version constant belongs in the upgrade runbook (w1-admin-authority, w2-upgrade-linkage).
- PackageVersion::assert_current precedes every presale participant exit (blast_presale.move:819, 856, 879); upgrade sequencing must publish new bytecode before bumping the version or all exits stall until the upgrade lands (w1-presale-settle).

• A `MigrationReady` launch whose venue permanently refuses (Cetus pool-manager pre-creation `cetus.move:15-17`, de-registered tier `:104-106`, or `VaultRegistry` issuer/venue removal `blast_vault_package.move:20-24`) has no product-side release of traders' Quote except a package upgrade; `buy` and `sell` are closed (`launch.move:694, :721-724`) and the migration pause bit only halts. A time-gated permissionless pro-rata refund or an admin-signed fallback returning the reserve to the curve would bound the lock without trusting the venue (w1-launch-migrate).

• A presale migration pause held for a full liquidity window converts any Funded sale into a cancellable, refund-only sale with its minted supply stranded (`blast_presale.move:933, 851-865, 1002-1003`); worth stating in the pause runbook as an accepted `PauseAdmin` authority (w1-presale-settle).

• After the liquidity deadline, `migrate` (no upper time bound, `blast_presale.move:924-935`) and `cancel_unlaunched` (`:851-865`) race and PTB ordering decides the outcome (pinned by `blast_presale_tests.move:1607-1619`); if refunds should be guaranteed once the deadline passes, `migrate` would need an upper bound; otherwise document the race for participants (w1-presale-settle).

• Cetus bytecode retirement window: the migrators link to Cetus v15 by `published-at` (`sui/migrators/interfaces/cetus/Move.toml:4`); when Cetus upgrades and bumps its `GlobalConfig` package version, every v15-linked route aborts until a migrators relink lands after the 48-hour notice (`DEPENDENCIES.md:509-516`; `DEPLOYING.md:44, :49-56, :88-99`). Graduated launches wait in `MigrationReady` and LP fees stay uncollected for that window (w1-cetus).

• `blast_vault_package::remove_witness` (`:101-109`) takes effect at once; removing an issuer stalls every pending migration for that product until re-admission waits the applied 48 h notice (`deployments/README.md:98-103`). Document re-admission of an issuer as a 48 h outage for sales at `MigrationReady`, or provide an issuer-specific re-admission path with its own notice (w1-vault-owner-registry).

• Only `sui/vault/sources/blast_vault_package.move` wraps `make_immutable`; `sui/launchpad`, `sui/presale` and `sui/migrators` declare `PackageWitness` without any `make_immutable` path (migrators constructs no `PackageWitness` at all, `blast_migrators.move:25-27`), so those packages can never end their upgrades and a scheduled `MakeImmutable` on them would stay pending until cancelled. Not a vulnerability, but it contradicts the `package_admin` doc comment that each package wraps the call. Conversely the vault can retire the NV-1 hazard for good through `make_immutable` (`blast_vault_package.move:171`) (w1-admin-authority, w2-upgrade-linkage).

6.2 Delayed settings: zero post-release delays and unpause-guard coverage

`blast_admin::delayed::assert_delay` enforces only the 30-day ceiling (`delayed.move:254-256`), so any scheduling role may propose a next delay of zero that applies after one announced wait, after which that slot's changes are instant. `is_guarded` is consulted only at `unpause` (`launch_config.move:395-398`, `creation_policy.move:191-194`, `blast_vault_package.move:160-163`, `package_admin.move:266`) and never re-checked by ordinary operations. This matches Compound/OpenZeppelin timelock semantics and ADR 0134 (`:413-425`) accepts no floor. Consumers where this was noted (all admin-only, announced by the applied delay, not findings):

• Core `blast_treasury::schedule_destination` (`sui/core/sources/blast_treasury.move:32-42`): a zero next delay turns the treasury into a plain setter after one 48 h wait; both product unpause paths refuse while unguarded, but already-running products keep paying the applied destination. A structural floor (next delay >= initial treasury delay) would make the guard an invariant (w1-core).

• Launchpad config: any scheduling role may propose its own setting with next delay 0; the guard is enforced only at `launch_config::unpause` (`launch_config.move:384-401`). A minimum post-release delay floor in `delayed::schedule`, or an `is_guarded` recheck at apply, would make the guard continuous (w1-launch-config). Cross-role coupling: `TreasuryAdmin` lowering the treasury delay to zero makes `launch_config::unpause` (`:396`) refuse until it is raised again (instant at delay 0, so no lock-out); release checklists should treat the treasury notice as a launchpad release prerequisite (w1-launch-config).

• Per-launch cap reassignment slots (`launch.move:402, :476`): `schedule_curve_fee_cap_delay` / `schedule_creator_metadata_cap_delay` (`CreatorAdmin`) may propose zero after the applied 48 h; `launch_config::unpause` covers only `LaunchConfig`, `RuntimeConfig` and `Treasury`, not these slots. If creator notice is a product promise, add a nonzero floor or alert on `ActionScheduled` events with `delay_ms == 0`

(w1-launch-claims, w6-obvious-launch-presale, w1-admin-primitives).

• Presale `fee_policy::schedule_fees` (`sui/presale/sources/config/fee_policy.move:57-64`): after a zero next delay applies, later rate changes for new sales become immediate; monitor `DelayedScheduled` events with `delay_ms == 0` on the `FeePolicy` (w1-presale-create).

• Per-vault round publication delays (`holder_rewards.move:162-173`, `RewardsAdmin`-governed, pause-gated): a vault's round notice may be lowered to zero, after which `schedule_round` and `publish_round` fit in one PTB for that vault and the `CancelAdmin` brake no longer applies there; `blast_vault_package::unpause` does not enumerate per-vault slots. A nonzero floor for `RoundPublicationKey` delays or monitoring of `Change::Delay` to 0 would close the gap (w1-vault-rewards, w1-vault-owner-registry).

• Migrators `schedule_upgrade_delay` accepts zero (`delayed.move:254-255`) and nothing in migrators refuses it because the package has no pause bits or unpause guard; lowering the notice to shorten a relink window removes the 48 h guard on the only package that can borrow vault LP positions. Consider a nonzero floor or an explicit procedure (w2-upgrade-linkage).

• `DelayedMap` (`sui/admin/sources/delayed/delayed_map.move`) has no in-scope consumer; if one is adopted, its `is_guarded` (pending must be empty) must be wired into that consumer's unpause the way `DelayedSet` is today (w1-admin-primitives).

6.3 Admission heuristics for venue, issuer and witness types

• `launch_config::schedule_venue` (`launch_config.move:302-305`), `creation_policy::schedule_venue` (`creation_policy.move:72-75`) and `blast_vault_package::schedule_witness` (`blast_vault_package.move:62-76`) reject only primitive types and types whose original package is `0x1` or `0x2`. Other system packages (`0x3`, `0xb`, `0xdee9`) and any third-party type with a public `drop` constructor pass the structural check; the on-chain gate cannot verify constructor privacy, so admission safety rests on `VenueAdmin` review plus the admission delay, as the doc comments state. Extending the origin exclusion to every system package and documenting the check as a typo guard rather than a trust boundary would match the stated intent (w1-launch-config, w1-presale-create, w3-obvious-foundation, w1-vault-owner-registry).

• The `VaultRegistry` keeps venues and issuers in one witness set (`blast_vault_package.move:11-17, 185-198`), so an admitted venue witness would also pass `assert_package_witness` and an admitted product witness would also pass `assert_venue`. No outsider path exists because both product `PackageWitness` constructors are `public(package)` and neither venue seals terms under `Venue()`, and source shows no lower-trust exploitation path even under mis-admission; two separate sets (or a role tag per member) would remove reliance on that discipline, with an admission review checklist in `deployments/README.md` step 1 (w1-core, w1-vault-owner-registry).

• `migration::new` accepts `venue: TypeName` as plain data and `migration::vault_terms` is public, so a `Migration` proves only that terms were sealed by the named witness; the load-bearing control that rejects outsider-sealed terms is `vault::new`'s `registry.assert_package_witness` (`vault.move:130`), executed before any venue state is shared. Any future venue must keep `vault::new` in the same transaction and before irreversible side effects (w1-core).

6.4 Functions without the version gate (uniformity only)

All read-only or same-principal; stale bytecode gains no privilege. Adding the gate would keep the "version gate at every mutator" invariant literal:

• `vault::share_with_position` (`vault.move:183-218`), the one vault state mutator without `assert_current`; harmless because the key-only, store-less `Vault` it consumes can only come from the gated `new` in the same PTB (w3-obvious-foundation).

• `vault::has_auto_lp_position` (`:233`) and `auto_lp_position` (`:254`), venue-keyed read-only borrows whose mutable siblings are gated (`:227`, `:267`) (w3-obvious-foundation, w1-vault-positions).

• `launch_config::venue_admission` (`:406-413`) and `creation_policy::venue_admission` (`:202-209`): public readers without `assert_current` or a pause check; they issue a droppable `VenueAdmission` that still requires the venue witness and is re-checked by core (w1-launch-config, w1-presale-create, w6-obvious-launch-presale).

• `blast_presale::destroy_cap` (`:1004-1010`) burns the caller's own cap without `assert_current` (w1-presale-create).

• `blast_presale::finalize` omits the create-pause-bit check that `launch::finalize` performs; equivalent because `PresaleInitializer` is a hot potato issued by `blast_presale::new` in the same transaction after checking the bit (w6-obvious-launch-presale).

• `blast_presale::prove_owner` (`:975-984`) has no lifecycle gate; harmless because the proof binds to the presale UID and package witness and is consumed in-transaction; a Migrated-only check would narrow the surface (w1-wild-presale).

6.5 Launch creation and migration economics

• **Creation-time seedability check missing in** `launch::finalize` (`launch.move:221-336, :264-266`): it does not call `migration::is_seedable_price` on `(lp_meme, lp_quote)` the way `blast_presale` does (`blast_presale.move:519, :711, :1234-1241`). With the default admitted decimals `{6, 9}` (`launch_config.move:488`) the bound is unreachable, but admitting decimals 10 via `set_meme_decimals_allowed` (`launch_config.move:148-160`) would let plans with `lp_quote` in `{1, 2}` atoms reach `MigrationReady` and fail the venue's `is_seedable_price` forever; stranded value is bounded by `graduation_target <= 29,999` Quote atoms plus the creator's own Meme. Assert seedability after `launch.move:270`; regression: a 10-decimals ticket with `lp_quote_bps = 1` and `graduation_target = 10_000` must abort at finalize (w1-launch-migrate, w1-launch-create).

• `set_meme_decimals_allowed` accepts any `u8`, while `launch_ticket::atomic_supply` (`launch_ticket.move:81-83`) overflows and aborts for decimals >= 11; bounding decimals at the setter (<= 10) turns a runtime abort into an admin-time rejection (w1-launch-config).

• **No LP-share floor:** `allocation::plan` (`allocation.move:52-55`) accepts `lp_meme_bps = 1` and `lp_quote_bps = 1`, so a creator may route up to `10,000 - protocol - 1` bps of the final Quote reserve to its own recipients and seed the pool with dust; disclosed in `LaunchCreated` (`events.move:76-104`) and the creator's own choice, so product policy rather than a boundary failure. A minimum LP share constant would remove the rug-shaped configuration (w1-launch-migrate, w1-launch-create).

• `launch::migrate` reads Blast's LP-fee share live from `LaunchConfig` at migration (`launch.move:844`, documented at `launch_config.move:37-38, :110-111`), whereas `blast_presale` snapshots it at creation; both are `FeeAdmin`-controlled, delayed and capped at 3,000 bps. The creator never sees the applied LP share (`launch_created` omits it) and a `FeeAdmin` may raise it before a `MigrationReady` launch migrates. Either snapshot the share at finalize into `Launch` or emit the applied share in `launch_created` (w1-launch-migrate, w6-obvious-launch-presale).

• `launch_ticket::destroy` (`launch_ticket.move:113-123`) fixes the supply on whichever `&mut Currency<Meme>` the launcher supplies without binding it to the ticket's cap IDs; safe today only because the framework's non-OTW `coin_registry::new_currency<T>` requires `T: key` while the ticket's Meme is an OTW with exactly `drop`, so no second `Currency<Meme>` can exist. A one-line defence in depth, `assert!(currency.treasury_cap_id() == option::some(object::id(&treasury_cap)))`, would make the fixed-supply guarantee independent of that framework property; the same binding is worth checking on `blast_presale::finalize` (w1-launch-create).

• `curve::cap_buy_quote_input` (`curve.move:25-28`): the `u64` `mul_div_ceil` can abort only when `graduation_target - real_quote_reserve` exceeds about `0.9 * u64::MAX` and a single buyer supplies more net Quote than that; unreachable for any real Quote supply and transaction-local, but the creation surface could cap `graduation_target` so the branch is provably overflow-free (w1-launch-trade).

• `launch::migrate` disposes of unsold Meme and pending sell-burn under a fixed supply by transferring to `@0x0`; the `Migrated` event documents this, but the coins remain counted in `total_supply`, which integrators reading supply should expect (w6-obvious-launch-presale).

6.6 Creator caps, attestations and recipients

• `apply_creator_metadata_cap_reassignment` (`launch.move:529`) clears `creator` exactly like the curve-fee path (`:455`). For a launch whose creator has not yet claimed, a metadata-only administrative fix therefore also revokes the creator's right to self-claim the `CurveFeeCap`, leaving `curve_fee_cap_id` none (`:315`) and creator fees (`:67-68`) unclaimable until `CreatorAdmin` schedules a second 48 h reassignment. Consider clearing `creator` only when the curve-fee cap is reassigned, or minting the missing sibling cap to the same recipient in the same apply (w1-launch-claims).

• `creator_authority.move:104` enforces only `now <= expiry_ms`; there is no on-chain bound on attestation lifetime and no per-attestation revocation other than revoking the whole signer key. Backend policy should issue short-lived attestations (w1-launch-claims).

• `runtime_config.move:35-58`: readmitting a previously revoked signer key re-enables every unexpired attestation that key ever signed. Rotate to a fresh key after a suspected leak and never readmit a revoked key (w1-launch-claims).

• Cap reassignment (`launch.move:460, :534`) and presale token/raise recipients (`blast_presale.move:235-237, 1147-1155`) accept any non-zero address, including an object address (or the presale's own derivable address), from which coins are unrecoverable or receivable only by that object's module. Same-principal misconfiguration; creation and admin tooling should validate recipients are account addresses (w1-launch-claims, w1-wild-presale).

• `set_name` / `set_description` / `set_icon_url` (`launch.move:543-586`) impose no length bound beyond the framework `Currency` behaviour and re-emit the complete fields in `MemeMetadataUpdated`; a metadata-cap holder who stores a very large icon URL can make later updates exceed the event size limit for that coin only (self-inflicted) (w1-launch-claims, w1-wild-launchpad).

6.7 Presale terms and lifecycle

• `blast_presale::finalize` bounds neither `commitment_duration_ms` nor `liquidity_window_ms` above (`blast_presale.move:487-491`), and `payout::vesting_terms` permits schedules up to `2^63` ms. Disclosed sale terms, not a protocol invariant failure, but creation tooling or a product-level maximum should bound these windows to protect participants who rely on UI summaries (w1-presale-create).

• Under a hard cap while caps still apply (`:1487` vs `:1512-1515`), the offer is clamped to the raise remainder but not to the wallet's remaining cap, so a wallet offering more than `cap - committed` aborts `wallet_cap_exceeded` instead of receiving change. Usability only (w1-presale-commit).

• Once an `Anyone`-access uncapped phase opens (`:1416-1419`, `:1464-1475`, with `:216-217`), allowlisted wallets that have not yet committed share the phase's participant bound with everyone; `max_participants >= leaf_count` (`:1180-1184`) does not reserve slots for leaves. Matches documented semantics; creation tooling should document it (w1-presale-commit, w1-wild-presale).

• The last relayed claim receives up to `(participants - 1)` atomic units of rounding remainder per asset (`:1574, :1583`), and any relayer can choose which participant claims last; documented dust (w1-presale-settle).

• Deployment-time manifest discipline: the on-chain allowlist leaf uses the caller's index both as payload and as tree position (`:1548-1550`), so the off-chain builder must place leaf `i` at position `i`, supply `leaf_count` equal to the real leaf count, and avoid duplicate wallets (w1-presale-commit).

• `assert_unregulated_currency_bytes` (`:1318-1336`) hard-codes the `Currency` field order and the `SupplyState` / `RegulatedState` variant tags of framework rev `06734f6f`; a layout change would silently reject every sale rather than admit a regulated one. The source already instructs rechecking on a Sui pin change (w1-presale-create, w6-obvious-launch-presale).

• Linked template: `sui/presale_coin_template/sources/presale_coin.move:24, :49` hard-codes `METADATA_CAP_RECIPIENT = @0x3e7a`; creation tooling must substitute it or the sale token's metadata control (name, description, icon URL under the pinned registry) goes to a fixed address (w1-presale-create).

6.8 Vault fees, rewards and registry

• `vault::apply_protocol_bps` (`vault.move:536-545`) is permissionless and applies the new Blast share without first collecting the pool fees accrued under the old share, whereas the fee split is applied only after `settle_fees` via the venue (`cetus.move:227-240`, `bluefin.move:169-173`; `vault.move:334-344`). Bounded (<= 3,000 bps, 48 h delay, documented at `vault.move:518-519`, anyone may collect first); a venue-mediated `settle_and_apply_protocol_bps` would make the two settings symmetric (w1-vault-positions, w1-vault-owner-registry).

• `holder_rewards::settle_gas_reserve` (`holder_rewards.move:125, :271`) pays the publish-time treasury snapshot without consulting the live `Treasury`. The freeze is documented (ADR 0134:128-131) and exposure is bounded (<= 10% of the round's holder Quote, amount chosen by `RewardsAdmin`), but if the Treasury destination is rotated away because it was compromised, a routine settlement still pays the old address. Consider taking `&Treasury` and asserting `self.treasury == treasury.destination()`, or paying the live destination (w1-vault-rewards).

• `claim_window_duration_ms` (`vault.move:616`) has only a `> 0` lower bound. A `RewardsAdmin` typo (or a compromised admin inside the 2-hour notice) can publish a round whose deadline is centuries away, locking that round's budget with no `close_round` recovery. Consider an upper bound (for example `delayed::max_delay_ms` or a product ceiling such as 90 days) checked in `new_round_terms` (w1-vault-rewards).

• `keep_auto_lp` (`vault.move:387-388`) records `added` as taken minus returned and cannot verify the venue actually deposited the difference into the pool position; the vault relies on the venue module being Blast-reviewed code (`cetus.move:449-458`, `bluefin.move:364-373`). Documented trust boundary (w1-vault-positions).

• `merkle::leaf` (`sui/core/sources/merkle.move:12-18`) does not length-prefix the domain. Current consumers are safe (domains are not prefixes of each other and payload lengths differ), but ULEB128 or fixed-length domain framing would make cross-consumer separation hold for any future domain pair (w1-vault-rewards).

• `payout::pay` (`payout.move:103-112`) and `pay_all` (`:115-119`) do not reject `@0x0` at the sink; zero-address rejection lives upstream (`recipient :47`, `fee_split::address_destination :40`, `blast_treasury :39, :83`, `holder_rewards::claim :219`). Intentional because `vault::burn` (`vault.move:804-809`) deliberately sends fixed-supply Meme to `@0x0`; a dedicated burn helper would let `pay`/`pay_all` reject zero defensively (w3-obvious-foundation).

• Linked dependency behaviour: a burn route on a collection that carries Quote but zero Meme reaches `route_fees` (`vault.move:798-801`) → `burn` (`:805-809`) → framework `coin_registry::burn_balance` with a zero balance; the framework source is not vendored, so the zero-balance behaviour was not verified from source (w1-vault-positions).

6.9 Venue adapters and linked dependency behaviour

• `bluefin::migrate` (`bluefin.move:105-107`) passes the Quote `Currency`'s live `symbol()` and `icon_url()` bytes to `create_pool_and_get_object` with no length bound, while the Meme icon is deliberately empty (`:99-101`). An admitted Quote issuer growing its icon URL could make Bluefin pool creation exceed object limits and block every Bluefin migration in that Quote until a migrators upgrade. The Quote issuer is a trusted party per ADR 0088:185-189; consider passing an empty Quote icon URL as for Meme (w6-obvious-migrators, w1-bluefin).

• Defensive closure of the prepare/migrate symbol check-use gap: for presale tokens, requiring the `MetadataCap` to be deleted (`presale_coin_template METADATA_MUTABLE = false`) or re-checking `currency.symbol().length() <= 32` inside `bluefin::migrate` would make the bound independent of the framework's setter surface. Under the pinned registry no symbol setter exists (see 7.3), so this is a latent assumption, not a defect (w1-bluefin).

• Irrevocable preparation before product validation: because `migration::redeem_ticket` (`sui/core/sources/migration/migration.move:79`) is public, a sealed-cap holder can run a venue `prepare` (`cetus.move:75-90`, `bluefin.move:58`) and consume the `VenueTicket` in the same transaction without any product `new`, permanently committing the write-once `VenueConfig` records (and, for Cetus, minting the Meme type's single `PoolCreationCap` and registering the pair). `cetus::prepare` also runs before the product's `new` validates

the Quote and venue (launch.move:126-127). A later prepare for that Meme on that venue aborts, so the coin can never sell through it. Only the coin's own sealed-cap holder is affected; clients should compose prepare and new in one PTB, and a product-witness-gated ticket consumer would remove the seam (w6-obvious-migrators, w1-cetus).

• The migrators package defines no pause bits (blast_migrators.move:8-13), so prepare, collect_fees, settle_and_apply_fee_split and compound_auto_lp cannot be stopped except by a 48-hour delayed version advance. Documented residual risk consistent with the stated design (w6-obvious-migrators).

• Seed-side fallback margin: cetus::seed_fixed_side (cetus.move:329-337) flips to the non-preferred side without re-running clmm_math::get_liquidity_by_amount. Under Cetus's rounding the two overdraw conditions are mutually exclusive by a margin of 2*(P_lo/2^64)^2 per atom. Re-checking the flipped side and aborting with a package error would make any failure attributable (w1-cetus).

• Empty auto-LP position ordering: cetus::add_liquidity (:439-447) opens and attaches the auto-LP Position before fixed_side decides whether anything is added, so a first two-sided compound with a few atoms leaves a zero-liquidity position that collect_position_fees (:386-391) then collects with recalculate = true. No abort is expected; moving the attach after the fit test removes the case (w1-cetus).

• Extreme-tick compound abort: cetus::fixed_side (:466-491) calls get_liquidity_by_amount fixed-A and fixed-B unconditionally; Cetus aborts when the current tick is at or beyond the full-range bound. Economically unreachable, affects only compound_auto_lp, and clears when the price returns; guarding on pool.current_tick_index() against the position's range would remove the venue dependency (w1-cetus).

• Bluefin dependency assumptions carried by the contract: (a) Bluefin's acceptance of every tier inside Blast's bounds (1..=400, 1..=20_000; constants.move:14-22, DEPENDENCIES.md:481-483) is asserted from owner disassembly; the tier is write-once per Meme type and a refused tier strands the sale. (b) The auto-LP estimate's 2-atom margin assumes Bluefin floors liquidity and rounds the paired amount up once (Cetus-form math); if Bluefin rounded liquidity up, compound_auto_lp could abort at high sqrt prices (retryable, no fund loss). (c) ADR 0088:173-178 accepts that the anonymous migrate caller becomes the Bluefin pool manager, resting on the revision-17 fact that the manager role reaches only reward emissions and role hand-off (w1-bluefin).

6.10 Admin package specifics

• AdminCap has key + store (blast_acl.move:39-41); a holder can public_share or freeze it, after which anyone can sign_in with its roles. This is the holder's own delegation and revoke_admin ends it immediately, but runbooks should monitor AdminCap ownership changes (w1-wild-foundation).

• destroy_admin_cap is ungated by the root and consumes the holder's own cap; a thief of a transferable AdminCap can erase its table entry, which is equivalent to a revocation and leaves no residual authority (w1-admin-authority).

• grant_role accepts any type as Role, including primitives and framework types; such grants are inert because no scoped consumer signature names them. A non-primitive check would mirror package_version::new_publication (w1-admin-authority).

• A ready Upgrade or MakeImmutable action has no expiry (package_admin.move); once its deadline passes it can be executed at any later time until cancelled or replaced. Owners relying on the 48 h public notice may want a bounded grace window so a long-stale digest cannot be executed without a fresh ActionScheduled event (w1-admin-authority).

• schedule_upgrade binds only the 32-byte digest; the on-chain binding is complete, but the public review window is only useful if the owner publishes the source commit and reproducible build that yields each scheduled digest alongside the ActionScheduled<UpgradeKey<OTW>, UpgradeAction> event (w1-admin-authority).

• UpgradeAdmin<OTW> alone schedules, authorizes and commits; only CancelAdmin<OTW> can veto within the delay. This single-role plus timelock threshold is the designed policy; hold the two caps under distinct custody so the veto is meaningful (w1-admin-authority).

• valid_pause_bits and package_witness are fixed at publication with no setter (package_version.move:30, :39, :158), so a compatible upgrade cannot introduce a new pause bit for new operations; reserve bits now or

accept reusing existing bits (w1-admin-primitives).

• `Publication` can only be transferred by `keep` to the init sender and consumed by `package_admin::wrap`, which also needs the `UpgradeCap` (`package_version.move:77-79`); until `wrap` runs no `PackageVersion` exists and every version-gated call is unreachable. Fail-closed, but the publish runbook must wrap in the same custody session (w1-admin-primitives).

• The same address `0x818d…d0a2` is compiled as `blast_acl_root` (`sui/admin/Move.toml:13`, receives the cold `SuperAdminCap`) and as `blast_treasury` (`sui/core/Move.toml:16`, receives every protocol fee payout); both inits abort on `0x0` (`blast_acl.move:54`; `blast_treasury.move:83`). Routing hot fee flows to the cold-root wallet couples root custody with routine treasury operations; the destination can be moved only after the compiled 48 h delay (core `constants.move:49-53`). Confirm this is a multisig and consider distinct addresses (w1-wild-foundation, w3-obvious-foundation).

• Dependency package `sui/partners` (out of scope for findings): `cancel_share` and `cancel_cap` require `PartnerAdmin` rather than a separate `CancelAdmin` (`blast_partners.move:107, 136`), so a compromised `PartnerAdmin`'s pending share raise is cancelled only after root revokes and re-grants the role (w1-wild-foundation).

• Role-scope drift: `blast_domain::TreasuryAdmin` is documented as "Chooses where Blast is paid" and `FeeAdmin` as "Sets Blast's fees and shares" (`sui/core/sources/blast_domain.move:15-19`), yet `schedule_protocol_allocation` (Blast's Meme/Quote allocation share, <= 1,000 bps, delayed) is gated on `AdminWitness<TreasuryAdmin>` (`launch_config.move:73-86`). An operator granting `TreasuryAdmin` for destination changes also grants an economic lever; consider moving it to `FeeAdmin` in the next launchpad upgrade and documenting the role (w1-wild-foundation, w1-launch-config).

6.11 Events and indexer notes

• `blast_admin_events::VersionAdvanced` / `UpgradeCommitted` / `UpgradeCapDestroyed` carry `original_package_id` as `address` while `PackageAdminCreated` carries it as `ID` (`blast_admin_events.move:48, 66, 85, 94`); harmless on chain but indexers must decode both (w1-wild-foundation).

• `CreatorCapsClaimed` (`sui/launchpad/sources/lib/events.move:261-264`) carries no `launch_id`, unlike every other launch event; consumers must key it by the Meme type parameter (one launch per Meme is structural via `LaunchKey`) (w1-wild-launchpad, w1-launch-claims).

• `launch::migrate` (`launch.move:828-918`) has no `public_start_ms` gate, unlike `sell` and `claim_creator_caps`, so a creator can buy to the target and fully migrate a launch (venue pool and vault included) inside the creation PTB before `share`; ADR 0090 accepts this and no outside party can hold funds in such a launch, but indexers must tolerate `Bought`, `MigrationAllocationPaid` and `Migrated` preceding `LaunchShared` (w1-wild-launchpad, w1-launch-trade, w1-launch-create).

• `launch::buy` (`launch.move:683-707`) returns a zero-value `Coin<Quote>` refund in the common non-capped path and a zero-value `Coin<Meme>` in the terminal zero-output path; PTB builders must consume these coins explicitly (w1-launch-trade).

• Presale `Committed` (`sui/presale/sources/lib/events.move:59-73, 352-358`) carries no committer address; indexers must attribute commits from the transaction sender, which is ambiguous for relayed or sponsored transactions (w1-presale-commit, w1-wild-presale).

6.12 Documentation drift (contract semantics)

• `sui/launch_ticket_template/README.md:28-31` and ADR 0088:184-185 state that launch intake/finalize rejects a Meme symbol longer than 32 bytes, but `launch::finalize` (`launch.move:221-336`) has no symbol check; the bound is enforced only in `bluefin::prepare` (`bluefin.move:53-56`). Either update the documents or add a venue-neutral bound in `launch_ticket::new` so a future venue that stores the symbol inherits it (w1-wild-launchpad).

• ADR 0137 lists `AdapterAdmin` where code declares `VenueAdmin`; ADR 0097 still says the root cap goes to the publisher and mentions a 32-admin total cap and the util timelock that ADR 0134 removed (w1-wild-foundation).

• ADR 0105 still describes `RuntimeConfig.treasury` and `set_treasury`, which ADR 0137 replaced with the core `Treasury` (w1-wild-launchpad).

• ADR 0106 (`:28-30`, `:57-61`) is stale: current code pays the partner a cut of every held asset (`protocol_fees.move:100-116`) and `pay_protocol_fees_with_partner` sends nothing instead of aborting when the ledger is empty (w1-wild-presale).

• `sui/core/sources/merkle.move:11` doc comment cites `b"blast-holder-rewards-v1"` while the vault hashes under `b"blast-holder-rewards-v2"` (`sui/vault/sources/lib/constants.move:61`) (w1-vault-rewards).

6.13 Positive source patterns

Patterns the hunters and verifiers confirmed from source and that calibrate the posture summary:

• **Live role table, not possession:** `blast_acl::sign_in` checks a table entry per `AdminCap` ID and role `TypeName` (`with_defining_ids`); `SuperAdminCap` revocation is immediate; root transfer is a 7-day `DelayedAction`.

• **Cross-package sealing by original IDs:** `migration::new` / `redeem`, `sealed_treasury_cap` and `owner_proof` compare `with_original_ids` type names of product `PackageWitness` and venue `Venue` witnesses; `Migration`, `VenueTicket`, `OwnerProof` and `PresaleInitializer` are no-ability hot potatoes, so a venue abort rolls the whole PTB back and a Funded sale stays Funded.

• **Object binding:** cap IDs are stored in the sale; receipts carry `vault_id` and cap returned <= taken; venue collection checks `pool_id`; rounds check `vault_id`; one launch or presale per coin type and one vault per (issuer, origin) via `derived_object::claim`.

• **Supply custody:** the sale token's `TreasuryCap` is sealed under the presale `PackageWitness` from birth and consumed into `SupplyState::BurnOnly` or `Fixed` at `finalize` (`blast_presale.move:1354-1355`); `launch_ticket` mints the atomic supply once and seals the cap.

• **Compiled economic ceilings:** bps <= 10,000, trade fees <= 10%, protocol share <= 70%, allocation <= 10%, LP protocol share <= 30%, partner <= 80%, creation fee <= 10 SUI, sell burn <= 50%, bounded list sizes (16 / 64 / 20 / 4 / 65,536).

• **Guard ordering:** version gate first, then pause bit, then lifecycle enum; delays bind to the applied value; `unpause` refuses while any covered notice is unguarded; cancel and remove paths intentionally work while paused (`vault.move:548-556`, `blast_vault_package.move:90-98, :101-109, :136-144`) so cancellation is never blocked.

• **Presale accounting invariants:** `total_commitments == quote.value() == sum of Commitment.quote` with every record `quote >= 1` (`blast_presale.move:1409, 1421-1428`); the hard-cap minimum waiver (`:1403`, `:1412-1415`) admits a below-minimum participant only on the commit that completes the raise and never waives the participant bound or the cap; lifecycle is one-way (Open → Funded → Migrated, or → Cancelled); cancellation refunds every commitment in full and pays Blast nothing.

• **Opening-round delivery** shifts at most one atom per asset per bidder because of floor rounding (`launch.move:656-678`); documented and not exploitable for value.

• **ADR 0088's "symbol fixed at creation" holds** for launches and presales alike under the pinned framework (see 7.3).

7. Coverage summary

7.1 Ledger

Status	Units	Notes
Total ledger units	62	54 seeded, plus six production-contract "Obvious things" units and the two Wildcard units reopened under a fresh owner
Effective in-scope contract units	55	denominator for coverage; excludes the 7 out-of-scope units
`covered`	52	every check `method: "source"`, `artifact: null`
`candidate`	3	linked to the two fingerprints below; all resolved in Phase 3 and verified in Phase 5
`blocked`	0	
`deferred`	0	25 units deferred for the first credit failure and 3 for the usage limit were all reopened and completed
`out_of_scope`	7	six repository-hygiene units superseded under the user's scope rule, plus the external `sui/vesting` published-pin item (see 1.6)

The three `candidate` units and their resolutions:

Candidate unit (surface :: boundary :: class)	Linked fingerprint	Phase 3	Phase 5
`sui/vault/sources/vault.move#assert_current gate on venue-only mutators versus migrators package linkage` :: `sui/admin/sources/package/package_admin.move#commit_upgrade_and_advance version bump ordering against dependent republication (deployments runbook)` :: `RESOURCE-EXHAUSTION-AND-AVAILABILITY.md#Unsafe recovery and capacity rollback`	`blast-v2/vault/vault/venue-gate-version-advance-uncoupled-from-migrators-relink`	`needs_validation` (fresh verifier `v6-vault-version-relink`)	verified unchanged (`f6-vault-version-relink`)
Candidate unit (surface :: boundary :: class)	Linked fingerprint	Phase 3	Phase 5
`sui/migrators/sources/venues/bluefin.move#prepare,migrate,collect_fees,settle_and_apply_fee_split,compound_auto_lp,buyback_and_burn` :: `…#is_meme_seed_side and add_liquidity_with_fixed_amount seeding, auto_lp_fixed_side can_fix_a/can_fix_b with the 2-atom margin and u128 liquidity bound, swap a2b=false by_amount_in with amount_limit 0 and max sqrt price, tick spacing`	`blast-v2/migrators/bluefin/symbol-bound-checked-at-prepare-only`	`rejected` (fresh verifier `v6-bluefin-symbol`)	verified unchanged (`f6-bluefin-symbol`)

Candidate unit (surface :: boundary :: class)	Linked fingerprint	Phase 3	Phase 5
and fee rate bounds, symbol length :: RESOURCE-EXHAUSTION-AND-AVAILABILITY.md#Reachable fatal error or deadlock			
sui/migrators/sources#production Move modules :: sui/migrators/sources#obvious reachable contract defects :: ATTACK-CLASSES.md#Obvious things	cited blast-v2/migrators/bluefin/symbol-bound-checked-at-prepare-only by fingerprint without a new record	as above	as above

No ledger candidate is unresolved; none is presented as a finding.

7.2 Attack-class selection and important exclusions

Selected ordinary classes: Access control, Business logic, Cryptography and secrets, Chained vulnerabilities and trust boundaries, Wildcard, Obvious things. Selected companions: `RESOURCE-EXHAUSTION-AND-AVAILABILITY.md` (permissionless shared-object paths whose abort or stranded state affects other users' exits), `SUPPLY-CHAIN-AND-RELEASE.md` (upgrade authorization and digest binding in `package_admin`; venue interface pins versus live bytecode), `DATA-ISOLATION-AND-LIFECYCLE.md` (stale authorization after revocation or cap reassignment; round lifecycle).

Excluded with per-unit reasons recorded in the ledger: Injection (no string, SQL, HTML, shell, template, path, URL or log sink; the only external decoders are fixed-layout BCS reads reviewed under Cryptography and Business logic), Resource and file handling (no filesystem or deserialization surface; Move arithmetic aborts rather than wraps, and stranded state has dedicated units), Feature abuse and data leakage (no export/import/search/webhook features; all chain state is public), and the web, client-side, memory-safety, AI, cloud, protocols/RPC and desktop/IPC companions (no such surfaces in Move packages).

7.3 Candidate validation and the rejected record

Two candidates reached Phase 3; both were validated by fresh Fable/high verifiers independent of the original hunters and verified unchanged by separate fresh Phase 5 verifiers.

- `blast-v2/vault/vault/venue-gate-version-advance-uncoupled-from-migrators-relink` → `needs_validation` (Section 5, `NEEDS-VALIDATION.md`).
- `blast-v2/migrators/bluefin/symbol-bound-checked-at-prepare-only` → `rejected`. This fingerprint is mentioned only to explain the coverage decision. The hunter's hypothesis required that a presale creator holding the generated package's raw `MetadataCap<SaleToken>` could lengthen the `Currency` symbol after settlement so that `bluefin::migrate` (`bluefin.move:98, :102`), which re-reads the live symbol without repeating the 32-byte bound checked once in `prepare` (`:53-56`), would abort. The verifier read the exact pinned framework source (`sui::coin_registry` at `06734f6ff0af45d8632a14a4dc4b100197f6b1a2`, pinned by `sui/migrators/Move.toml:14`): the `MetadataCap` setters are `set_name`, `set_description` and `set_icon_url` only; no `set_symbol` exists; the only post-creation symbol writer, `update_from_legacy_metadata`, aborts unless the cap is neither claimed nor deleted, and every sale token's cap is marked `Claimed` or `Deleted` at `finalize`; the `TreasuryCap` a legacy `coin::update_symbol` would need is consumed by `blast_presale::finalize` (`blast_presale.move:1354-1355`); and the fields the cap can change never reach Bluefin (`bluefin.move:101` sends an empty Meme icon and passes no name or description). The symbol `bluefin::migrate` reads is therefore exactly the symbol `prepare` bounded; no lower-trust principal can induce the abort. Residual observations from that validation are recorded as hardening in 6.9 (Quote-side symbol and icon URL unbounded; defensive re-check of the symbol bound).

7.4 Critic results

- Post-wave coverage critic (`c6-postwave-critic`, Fable/high, read-only): no missing units, no reassignments, `stop: true`.

- Final-clean coverage critic (`c6-final-clean-critic-2`, Fable/high, read-only with the Bash deviation disclosed in 1.4): no missing units, no reassignments, `stop: true`. The earlier final-clean attempt was discarded and is not used as evidence.

Both mandatory critics ran. The clean-coverage claim is limited to the 55 effective in-scope contract units reviewed from source; it says nothing about runtime, deployment, harnesses, tooling or the out-of-scope units.

7.5 Record disposition

`findings.json` holds two records and both passed Phase 5 verification unchanged: one `needs_validation` (NV-1) and one `rejected`. Zero `confirmed`. Both skill validators (`validate-coverage-ledger.cjs`, `validate-findings.cjs`) pass.

Findings detail

Run `2026-11-06-fable-cloudflare-launch-stack`; `interest-protocol/blast-v2-contracts` at `d80f2834468ba798df66f36d01fd64eec9d4df91`; Cloudflare `security-audit` skill `c1c8a8c1471069fb0e188eeaff69b8e8db6564a8`; Claude Fable 5.1 (`claude-fable-5-1`), effort xhigh for reconnaissance and the first 12 hunters, high for the rest.

Confirmed `medium`, `high` or `critical` records

None.

`findings.json` holds no record with verdict `confirmed`. This run was source-only: no OS sandbox or race-safe artifact promotion was provisioned, so nothing was built, run, tested or probed, and the skill's execution-proof gate for `confirmed` could not be met by any candidate. There is therefore no repository trace, dummy principal, native invocation, observed output, containment statement or regression case to copy here. Nothing in this file should be read as a clean bill for runtime or deployment behaviour, which remain untested.

Where the records are

- One record is `needs_validation`: `blast-v2/vault/vault/venue-gate-version-advance-uncoupled-from-migrators-relink`. It has no severity and is not a confirmed vulnerability. Its complete source trace, verified evidence, exact blockers and bounded resolution plans are in `NEEDS-VALIDATION.md`.
- One record is `rejected` on source evidence; it is not a finding and is described only in `REPORT.md` section 7.3 to explain the coverage decision.

Both records passed fresh Phase 5 verification unchanged. Hardening notes and positive source patterns are in `REPORT.md` section 6.

Needs validation

Run `2026-11-06-fable-cloudflare-launch-stack`; `interest-protocol/blast-v2-contracts` at `d80f2834468ba798df66f36d01fd64eec9d4df91`; Cloudflare `security-audit` skill `c1c8a8c1471069fb0e188eeaff69b8e8db6564a8`; Claude Fable 5.1 (`claude-fable-5-1`) at effort high for the Phase 3 verifier (`v6-vault-version-relink`), the Phase 5 verifier (`f6-vault-version-relink`) and this document; the originating hunter (`w2-upgrade-linkage`) ran at high.

This file lists every unresolved record as a prioritized lead **without severity**. A lead is not a confirmed vulnerability. The run was source-only (no build, test, script, fixture, RPC or deployed probe), so the runtime and deployment facts named under "Blockers" are genuinely unknown; the plans below do not assume them. The local plan is an offline, loopback-only reproduction for the owners to run; it is not live test guidance.

There is exactly one lead.

NV-1. Vault version advance is uncoupled from the migrators relink; during the resulting venue outage any sender can irreversibly cancel a Funded presale whose liquidity deadline passes

- **Fingerprint:** `blast-v2/vault/vault/venue-gate-version-advance-uncoupled-from-migrators-relink`
- **Verdict:** `needs_validation` (Phase 3 fresh verifier `v6-vault-version-relink`; Phase 5 fresh verifier `f6-vault-version-relink`: verified unchanged)
- **Severity:** none assigned
- **Ledger unit:** `sui/vault/sources/vault.move#assert_current gate on venue-only mutators versus migrators package linkage` :: `sui/admin/sources/package/package_admin.move#commit_upgrade_and_advance version bump ordering against dependent republication (deployments runbook)` :: `RESOURCE-EXHAUSTION-AND-AVAILABILITY.md#Unsafe recovery and capacity rollback`
- **Affected boundary:** the presale lifecycle gate (`Funded` → `Cancelled`) as it depends on the vault's `PackageVersion<BLAST_VAULT_PACKAGE>` gate and the migrators package's static linkage to one vault generation. Packages involved: `sui/admin` (`package_admin`, `package_version`, `delayed_action`, `blast_acl`), `sui/vault` (`vault`, `blast_vault_package`), `sui/migrators` (`cetus`, `bluefin`, `blast_migrators`), `sui/presale` (`blast_presale`), `sui/core` (`migration`).
- **Lower-trust principal:** any transaction sender calling `blast_presale::cancel_unlaunched`. The principal fires a designed refund exit; it does not steal.
- **Privileged precondition:** the vault's `UpgradeAdmin<BLAST_VAULT_PACKAGE>` commits a compatible vault upgrade through `package_admin::commit_upgrade_and_advance`, which advances the shared vault version in the same call, before a relinked migrators upgrade has committed.
- **Character:** operator-triggered availability and recovery hazard, not a theft path.

Description (as verified)

Source confirms every link of the candidate. Every vault function a venue calls (`vault::new`, `position_mut`, `attach_auto_lp_position`, `auto_lp_position_mut`, `split_fees`, `apply_fee_split`, `take_auto_lp`, `keep_auto_lp`, `take_buyback_quote`, `burn_buyback`) first runs `package_version.assert_current(constants::version!())`, a macro literal inlined into the vault bytecode, and `assert_current` is strict equality against the shared `PackageVersion<BLAST_VAULT_PACKAGE>`. The migrators package links `blast_vault` as a dependency and reaches those functions only through the vault generation it was published against, while `blast_admin::package_admin::commit_upgrade_and_advance<BLAST_VAULT_PACKAGE>` takes only the vault's own `PackageAdmin`, `PackageVersion` and `UpgradeAdmin` witness and advances the stored version in the same call; no code reads the migrators package's upgrade state.

Once the vault commits, `cetus::migrate`, `bluefin::migrate`, `collect_fees`, `settle_and_apply_fee_split`, `compound_auto_lp` and `buyback_and_burn` abort with `EVersionMismatch` until a relinked migrators upgrade (new vault dependency ID plus a migrators version bump) commits; `cetus::prepare` and `bluefin::prepare` never touch the vault and keep working, so new sales can still be opened into the outage. `blast_presale::migrate` does not read the vault version but returns a no-ability `Migration` that only the stored venue witness can redeem, so the migration PTB aborts whole and the sale stays `Funded`. Its liquidity deadline is fixed at `share`, can only be brought forward by an early close, and has no extension path; `cancel_unlaunched` checks only the presale version, `Lifecycle::Funded` and the deadline, with no pause bit, so once the deadline passes inside the outage any sender cancels the sale.

Cancellation is one-way: `migrate` then aborts `EInvalidLifecycle`, participants are refunded their Quote in full, the raise recipients and token recipients receive nothing, Blast's raise and token fees are forgone, and the whole minted `SaleToken` supply stays locked in the cancelled presale (the `TreasuryCap` was consumed at `finalize`). The authors' runbooks acknowledge the outage and prescribe scheduling both upgrades together and executing them back to back, but the relinked migrators digest is computed over its modules plus dependency package IDs, and the vault's post-upgrade package ID is created by the vault upgrade transaction itself, so absent a precomputed ID or a pre-lowered migrators notice the relink can be scheduled only after the vault commits and is ready no sooner than the migrators' applied upgrade notice (48 h at publication) later. `CancelAdmin<BLAST_MIGRATORS>` can cancel a ready relink and restart the notice; the migrators package defines no pause bits and has no unpause, so nothing refuses lowering its notice to zero.

Exposure is bounded by the presale's 7-day minimum liquidity window: only Funded presales whose deadline falls inside the outage are affected, and whether the operators' release procedure ever opens such a window is a deployment fact.

Claimed root cause (unchanged from the record)

The vault's `assert_current` gate on venue-only mutators and `package_admin`'s one-call version advance are not coupled to the dependent migrators package: no code, pause bit or deadline extension protects in-flight Funded presales from the window between the vault's version advance and the migrators relink, and the runbook's back-to-back ordering is not shown to be achievable as written because the relink's upgrade digest depends on the vault's post-upgrade package ID, which exists only after the vault upgrade executes.

Source trace (repository-relative, ordered)

1. **Entrypoint** `sui/presale/sources/blast_presale.move:851`, `blast_presale::blast_presale::cancel_unlaunched`. Permissionless entry: any sender cancels a Funded presale once `clock >= liquidity_deadline_at_ms`. The function checks only the presale version (`:856`), `Lifecycle::Funded` (`:858`) and the deadline (`:859-862`); no presale pause bit is consulted, and no vault or migrators state is read. The remaining steps establish why the competing exit, migration, is unavailable during the window.

2. **Propagation** `sui/admin/sources/package/package_admin.move:165`, `blast_admin::package_admin::commit_upgrade_and_advance<BLAST_VAULT_PACKAGE>`. Privileged precondition: the vault's `UpgradeAdmin` commits a compatible vault upgrade and line 190 advances the shared vault `PackageVersion` in the same call. The signature takes only `PackageAdmin<OTW>`, `AdminWitness<UpgradeAdmin<OTW>>`, `PackageVersion<OTW>` and the `UpgradeReceipt`; `PackageAdmin<A>` and `PackageAdmin<B>` are independent shared

objects (`:43-61`), so the migrators relink state is never checked.

3. **Propagation** `sui/admin/sources/package/package_version.move:86`, `blast_admin::package_version::assert_current`. Strict equality between the stored version and the caller's compiled literal (`:86-89`); `advance` (`:233-243`) only increments, so no later vault action can restore the value old dependents compile.

4. **Propagation** `sui/vault/sources/vault.move:128`, `blast_vault::vault::new`. Every venue-only vault mutator inlines `constants::version!()` (`constants.move:10-12`, literal 1) and aborts `EVersionMismatch` once the stored version moved: `new` 128, `position_mut` 227, `attach_auto_lp_position` 244, `auto_lp_position_mut` 267, `split_fees` 289, `apply_fee_split` 340, `take_auto_lp` 353, `keep_auto_lp` 378, `take_buyback_quote` 413, `burn_buyback` 440.

5. **Propagation** `sui/migrators/sources/venues/cetus.move:170`, `blast_migrators::cetus::migrate`. The migrators package calls `vault::new` through the vault generation it links (`Move.toml:10`); `bluefin::migrate` (`bluefin.move:125`) and both venues' `collect_fees`, `settle_and_apply_fee_split`, `compound_auto_lp` and `buyback_and_burn` reach `position_mut`, `split_fees`, `apply_fee_split`, `take_auto_lp`, `keep_auto_lp`, `take_buyback_quote` and `burn_buyback` the same way (`cetus.move:240, 285, 288, 360, 380, 407, 420, 442, 445`; `bluefin.move:173, 213, 227, 330, 341, 359, 360, 378`). Only `cetus::prepare` (`:61-95`) and `bluefin::prepare` (`:39-63`) never touch the vault and keep working during the outage.

6. **Propagation** `sui/presale/sources/blast_presale.move:924`, `blast_presale::blast_presale::migrate`. Returns a no-ability `Migration` (core `migration.move:57-62`) redeemable only by the stored venue witness (`migration.move:137-142`), so a migration PTB whose venue call aborts rolls back whole and the sale stays Funded. `liquidity_deadline_at_ms` (`:1046-1048`) is fixed at `share` (`:622-628`), only shortened by an early close (`:830`), and has no extension path: `set_windows` (`:351-361`) exists only on the `PresaleInitializer`, before `finalize`.

7. **Sink** `sui/presale/sources/blast_presale.move:1662`, `blast_presale::blast_presale::cancel`. Lifecycle becomes `Cancelled` irreversibly (`:1663`); `migrate` then aborts `EInvalidLifecycle` (`:935`); `claim` (`:872-913`) refunds each participant its full Quote and zero tokens (`:1572-1573`, `:1585`); `destroy_cap` (`:1002-1010`) documents that every minted `SaleToken` allocation remains permanently locked in the cancelled presale, whose `TreasuryCap` was consumed at `finalize` (`:1340-1358`).

Verified evidence

File	Line	What it establishes
sui/admin/sources/package/package_admin.move	190	package_version.advance() runs in the same call as the native commit; no cross-package precondition exists, and lines 14-16 state that package_admin itself is never version gated, so a repair can always be scheduled.
sui/vault/sources/lib/constants.move	10	The vault version macro (literal 1) is inlined into vault bytecode, so the gate value is fixed per vault generation that dependents link.
sui/migrators/Move.toml	10	blast_vault is a local dependency of blast_migrators; the venues' vault calls resolve through this linkage.
sui/migrators/sources/venues/bluefin.move	125	bluefin::migrate reaches vault::new only through the linked vault generation.
sui/migrators/sources/blast_migrators.move	25	The migrators package defines no pause bits and nothing constructs its PackageWitness, so it has no unpause and no path that refuses a zero upgrade notice (package_admin::clear_pause_bits is the only is_guarded check, :258-269).
sui/presale/sources/blast_presale.move	351	set_windows is the only writer of liquidity_window_ms and exists only on the PresaleInitializer, consumed by finalize; settle (:830) can only shorten duration_ms on an early close, so no principal can extend a Funded sale's liquidity deadline.
sui/presale/sources/lib/constants.move	50	Minimum liquidity window is 7 days (enforced at finalize :487-491), bounding which Funded presales fall inside a 48 h-plus outage.
sui/migrators/DEPENDENCIES.md	524	Authors acknowledge that after the vault commits, migration and fee collection abort until the relink lands and prescribe scheduling both together and executing back to back; the presale consequence is not mentioned.
sui/migrators/DEPLOYING.md	95	The migrators runbook repeats that a vault upgrade requires a dependency-only relink in the same window and says to script both as one step, without describing how the relink digest is obtained before the vault's new package ID exists.
deployments/README.md	118	Release runbook repeats the same-window relink requirement without describing how the relink digest is obtained before the vault's new package ID exists.
sui/launchpad/ptb_tests/tests/atomic_migration.rs	609	Supporting evidence only: the authors' harness computes the upgrade digest with compute_digest_for_modules_and_deps over modules plus dependency IDs, passes those dependency IDs as a pure input to the Upgrade command (:664), and at :690-695 rebuilds the dependent only after substituting the upgraded package ID, showing the relink digest depends on the vault's post-upgrade ID.
sui/presale/tests/blast_presale_tests.move	412	Supporting evidence only: authors pin that cancel_unlaunched succeeds while every presale pause bit is set, so no admin pause shields a Funded sale during the outage.
sui/presale/tests/blast_presale_tests.move	1626	Supporting evidence only: authors pin that a cancelled unlaunched sale can never migrate.

File	Line	What it establishes
sui/admin/sources/package/package_admin.move	118	CancelAdmin<BLAST_MIGRATORS> cancels the scheduled relink even after it is ready (delayed_action.move:105-109), restarting the full notice on the next schedule (:67-74).
sui/admin/sources/blast_acl.move	112	SuperAdminCap role revocation is immediate, bounding a veto loop by root response plus one notice.
sui/admin/sources/delayed/delayed.move	254	assert_delay enforces only the 30-day ceiling, so the migrators notice may be lowered to zero (after waiting the applied notice, delayed_action.move:77-84) to shorten the window at the cost of the 48 h guard on the package that borrows LP positions.
sui/vault/sources/blast_vault_package.move	171	Calibration: the vault can end its own upgrades through make_immutable, which would retire this hazard for good; until then every vault upgrade reopens it.

Exact blockers (why this is not confirmed)

1. No sandboxed `cargo test` or `sui move test` run was permitted in this audit; a simulacrum harness run extending `atomic_migration.rs`'s upgrade flow to the vault, migrators and presale packages is needed to observe that `cetus`/`bluefin` `migrate` and `collect_fees` abort with `EVersionMismatch` at `blast_admin::package_version` after the vault's `commit_upgrade_and_advance` and recover only after a relinked migrators upgrade commits.

2. The Sui runtime rule that a dependent package keeps calling the dependency generation it was published against until it is itself upgraded is a framework fact outside the repository; the authors' docs and harness exercise it for a mock adapter and the launchpad, not for migrators → vault.

3. Whether the migrators relink can be scheduled before the vault upgrade executes (by deterministically precomputing the vault's post-upgrade package ID from a fully fixed transaction, and whether the relink's `Upgrade` command may name a dependency created earlier in the same transaction) or only after it, which decides whether the minimum outage is one transaction or at least the migrators' applied upgrade notice plus build time, is a toolchain/runtime fact outside source.

4. Deployment facts: the operators' actual release script for a vault upgrade, whether the migrators notice is pre-lowered for the release, which principals hold `CancelAdmin<BLAST_MIGRATORS>`, and whether any Funded presale's liquidity deadline falls inside the planned outage.

Resolution plans

Bounded local plan (offline, simulacrum, loopback only; in `sui/launchpad/ptb_tests`). Publish the stack, open a presale with the minimum 7-day liquidity window, fund and settle it; perform a compatible vault upgrade through `schedule_upgrade` / `authorize_upgrade` / `Upgrade` / `commit_upgrade_and_advance` with the vault version literal rewritten to 2 as `atomic_migration.rs:570-598` does for the launchpad; then:

1. assert `presale.migrate` + `bluefin::migrate` (or `cetus`) in one PTB rolls back with `EVersionMismatch` at `blast_admin::package_version` and the sale stays Funded, while `bluefin::prepare` still succeeds;

2. build the migrators relink with the vault dependency replaced by the new ID and assert its `compute_digest_for_modules_and_deps` differs from the pre-upgrade build's digest, and attempt to include the relink's `Upgrade` in the same PTB as the vault commit using the precomputed ID to learn whether a one-transaction window is achievable;

3. set all presale pause bits, advance the clock to the liquidity deadline, and assert `cancel_unlaunched` from an unrelated sender succeeds and `migrate` then aborts `EInvalidLifecycle`;

4. schedule the relink, cancel it with `CancelAdmin<BLAST_MIGRATORS>`, and assert a second schedule waits the full notice;

5. commit the relink and assert `bluefin::migrate` succeeds for a sale that was still Funded.

Owner-observed deployment check (read-only). Confirm from the release scripts and records whether a vault upgrade is ever executed before the relinked migrators digest is scheduled; how the relink digest is produced (precomputed package ID or post-upgrade build); the migrators' applied upgrade notice at release time; which AdminCaps hold `CancelAdmin<BLAST_MIGRATORS>`; and that no Funded presale will be within its last (notice + build time) of its liquidity window at the planned vault commit. Alternatively, confirm whether the vault is to be made immutable before any upgrade.

Related hardening notes (not findings)

The following hunter notes describe the same seam and are listed in `REPORT.md` section 6.1 and 6.2: `prepare` stays callable during the relink window (consider the products' create pause bits during a vault release); the runbooks should state how "schedule both together" is achieved; `CancelAdmin<BLAST_MIGRATORS>` is liveness-critical during vault releases; the migrators upgrade notice may be lowered to zero with nothing refusing it; a wrong vault version literal strands every vault path including direct exits for the repair window; a presale migration pause held for a full liquidity window has the same refund-only outcome by design; and `migrate` versus `cancel_unlaunched` race after the deadline with PTB ordering deciding.

Candidates that did not become leads

One other candidate reached Phase 3 and was **rejected** on the exact pinned framework source: `blast-v2/migrators/bluefin/symbol-bound-checked-at-prepare-only`. It is not a lead and not a finding; it is named here only so that the ledger's three `candidate` units (the vault/migrators linkage unit above, the Bluefin availability unit, and the migrators obvious-defect unit that cited the same fingerprint) are all accounted for. The reason for rejection is summarized in `REPORT.md` section 7.3.

No other ledger candidate is unresolved.